Memória compartilhada Mem0 — Claude Code + Codex (ai-kit)

Resumo

Kit global e reutilizável que dá ao Claude Code e ao Codex o mesmo espaço de memória no Mem0, um protocolo único de leitura e gravação seletiva, arquivos de contexto por projeto e relatórios PDF. Instala uma vez por máquina (`node install.mjs`) e se aplica a qualquer projeto, novo ou existente, com `node ~/.ai-kit/bin/ai-init.mjs` . A gravação, a leitura cruzada e o isolamento entre projetos foram testados de ponta a ponta nas duas ferramentas.

Objetivo

- Claude Code e Codex trocam aprendizados duradouros pelo Mem0, sem depender da conversa um do outro.
- Contexto operacional versionado no repositório e funcional mesmo sem o Mem0.
- Configuração de baixa manutenção para todos os projetos do usuário.

Implementação

Componente	Decisão
Integração Mem0	MCP hospedado `https://mcp.mem0.ai/mcp` (servidor `mem0` 1.29.1) nas duas ferramentas
Autenticação	Mesma chave `MEM0_API_KEY` (variável de ambiente do usuário) ⇒ mesmo projeto Mem0
Claude Code	MCP em escopo **user**; header `Authorization: Bearer ${MEM0_API_KEY}` expandido em tempo de execução
Codex	`[mcp_servers.mem0]` com `bearer_token_env_var = "MEM0_API_KEY"` em `~/.codex/config.toml`
Instruções	Fonte única `kit/AGENTS.global.md` → importada em `~/.claude/CLAUDE.md` e copiada para `~/.codex/AGENTS.md`
Escopo	`user_id=estevaols` ; `app_id=<projeto>` ou `global` ; `metadata.source` e `metadata.type`
Gravação	`add_memory` com `infer: false` (texto curado gravado como está), uma ideia por memória
Busca	Uma `search_memories` por tarefa quando necessária, `top_k` ≤ 5, filtro por `user_id` + `app_id in [projeto, global]`
Projetos	`ai-init.mjs` cria só o que falta; em arquivos existentes apenas anexa blocos marcados
PDF	`ai-report.mjs` : Markdown → HTML estilizado → PDF via Edge/Chrome headless, sem dependências

Alternativas descartadas

- **Plugins oficiais Mem0 (Claude Code e Codex)**: capturam automaticamente mensagens, respostas e arquivos via hooks, contrariando o requisito de gravação seletiva. A documentação proíbe combiná-los com o MCP manual (servidor duplicado).
- `.mcp.json` **por projeto**: exigiria repetir a configuração em cada repositório; substituído pelo escopo user.
- `agent_id` **para identificar o agente**: o Mem0 não grava `user_id` e `agent_id` no mesmo registro, então filtros com AND entre eles não retornam nada. Usado `metadata.source`.

Arquivos alterados

Arquivo	Descrição
`install.mjs`	Instalador idempotente (kit, instruções globais, MCP nas duas ferramentas, checagem da chave)
`kit/AGENTS.global.md`	Protocolo de memória e contexto (fonte única)
`kit/bin/ai-init.mjs`	Inicializa um projeto sem sobrescrever nada
`kit/bin/ai-report.mjs`	Gera o PDF de uma tarefa
`kit/bin/mem0-check.mjs`	Teste/diagnóstico do MCP Mem0 (nunca exibe a chave)
`kit/templates/*`	Modelos de `AGENTS.md`, `CLAUDE.md`, `AI_PROGRESS.md`, `AI_CONTEXT.md` e bloco ai-kit
`AGENTS.md`, `CLAUDE.md`, `AI_*.md`, `README.md`, `.gitignore`	Arquivos deste repositório

Fora do repositório (máquina do usuário): `~/.ai-kit/`, bloco em `~/.claude/CLAUDE.md` (conteúdo graphify preservado), `~/.codex/AGENTS.md` (novo), servidor `mem0` em `~/.claude.json` (escopo user) e em `~/.codex/config.toml` (a CLI do Codex reordenou o arquivo sem perder entradas).

Testes e resultados

Teste	Resultado
Endpoint sem chave	HTTP 401 `invalid_token` (esperado)
`mem0-check.mjs tools` com chave	Conectado, 11 ferramentas listadas
`claude mcp get mem0` (escopo user, chave no ambiente)	Connected
`codex mcp list`	`mem0` enabled, auth Bearer token
Claude Code headless (`claude -p`) grava memória fictícia	Gravou com `user_id=estevaols` , `app_id=meu-grande-prompt` , `metadata.source=claude-code` , `infer=false` , deduzidos só das instruções
Memória isca em `app_id=outro-projeto-teste`	Gravada; aparece em busca sem filtro de app
Codex headless (`codex exec`) busca pelo protocolo	Retornou apenas a memória do Claude Code; isca excluída pelo filtro
Limpeza	2 memórias e a entidade `outro-projeto-teste` removidas; `get_memories` vazio
`ai-init.mjs` em projeto novo e existente, 2×	Cria o que falta; anexa blocos uma única vez; não sobrescreve
`install.mjs` 2×	Idempotente
Pós-reinício: Claude Code interativo (`mcp__mem0__search_memories`)	Retornou as 2 memórias do projeto + global
Pós-reinício: app Codex (busca pelo protocolo)	Retornou as mesmas 2 memórias (IDs idênticos)
`ai-report.mjs` neste documento	PDF gerado e verificado visualmente

Limitações e pendências

- Isolamento entre projetos é por filtro, não por permissão: quem tem a chave lê todas as memórias da conta. Para clientes/projetos sensíveis, usar projeto Mem0 separado com outra chave.
- Memórias ficam na nuvem do Mem0; exclusão via `delete_memory` / `delete_entities` (testada). Retenção e privacidade seguem a política do Mem0.
- Claude Code e o app Codex precisam ser reiniciados após definir `MEM0_API_KEY` para enxergar a variável (feito e validado em 2026-10-28).
- Codex Cloud não foi configurado (exige `MEM0_API_KEY` como Environment Variable do ambiente na nuvem).
- A aderência ao protocolo depende do modelo seguir as instruções; não há hooks forçando gravação ou busca, por escolha.
- Em outras máquinas, rodar `node install.mjs` após clonar; o `user_id` está fixo em `kit/AGENTS.global.md` .